

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B. Charan kumar Reddy	Reg. No.:	192472195
Section / Batch:	2024/CSE-Ai	Date:	16/07/2026

Code of Conduct

I B. Charan Kumar Reddy (192472195) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.C.K.Reddy

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input
abaab

Output
Transition Path:
 $q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

- 1.Search Date
- 2.Search Phone Number
- 3.Search Hashtag
- 4.Search Mention
- 5.Search Prefix
- 6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO1-Assessment Tool 3

Name: B. Charan Kumar Reddy

Reg no.: 192472195

Course Code: DSA0303

Course Name: Natural Language Processing

Question 1: Intelligent Email and Password Validator using Regular Expressions

Aim

To develop a Python program using Regular Expressions (Regex) to validate an email address, password, and mobile number based on the given security rules.

Algorithm

1. Import the re module.
2. Read the email, password, and mobile number from the user.
3. Validate each input using Regular Expressions.
4. Display the validation results.
5. End the program.

Program

```
import re

def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return "Valid Email" if re.fullmatch(pattern, email) else "Invalid Email"

def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$'
    return "Strong Password" if re.fullmatch(pattern, password) else "Weak Password"

def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return "Valid Mobile Number" if re.fullmatch(pattern, mobile) else "Invalid Mobile Number"
```

```
print("==== Intelligent Validator ====")
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
print("\n----- Validation Result -----")
print(validate_email(email))
print(validate_password(password))
print(validate_mobile(mobile))
```

Sample Input

Enter Email: charan123@gmail.com

Enter Password: Charan@123

Enter Mobile Number: 9876543210

Sample Output

==== Intelligent Validator ====

----- Validation Result -----

Valid Email

Strong Password

Valid Mobile Number

Test Case 1

Input

Email: user123@gmail.com

Password: User@1234

Mobile: 9876543210

Output

Valid Email

Strong Password

Valid Mobile Number

Test Case 2

Input

Email: 123user@gmail.com

Password: user123

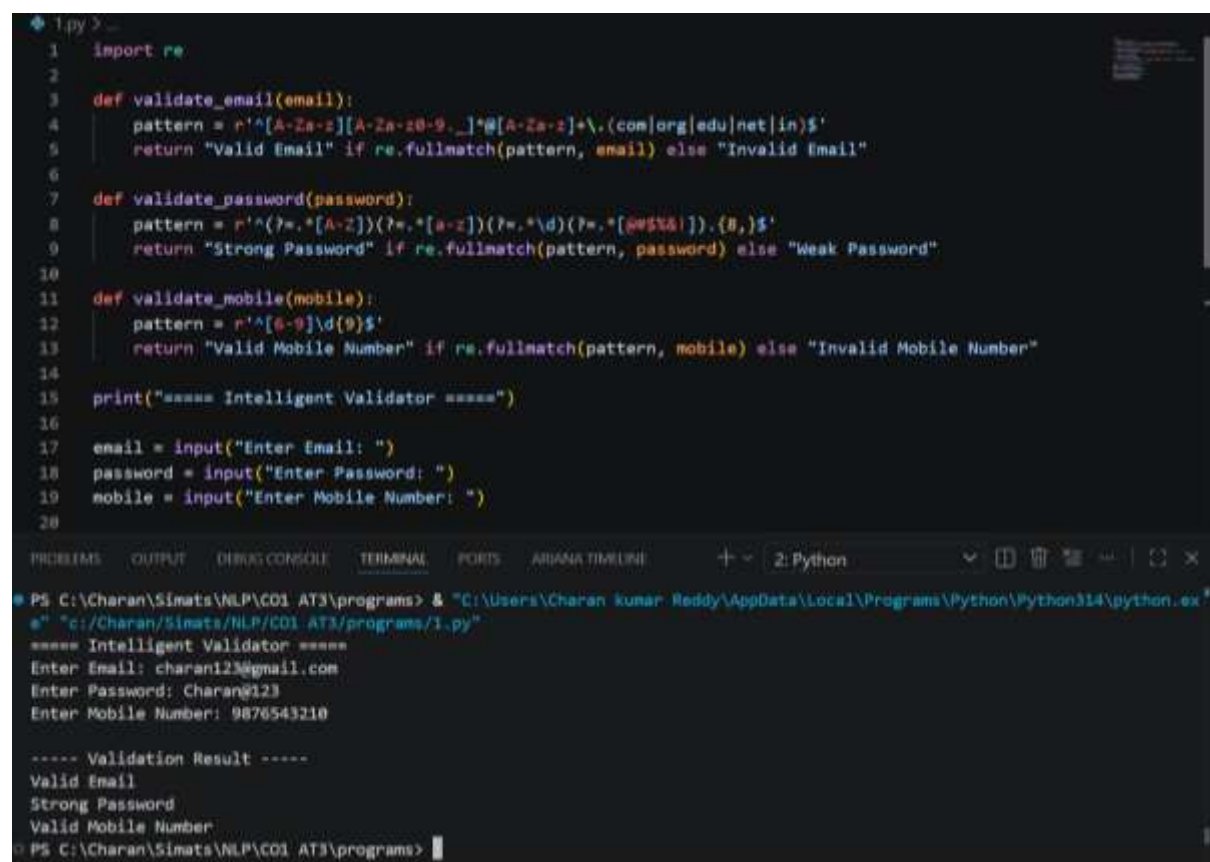
Mobile: 5876543210

Output

Invalid Email

Weak Password

Invalid Mobile Number

The image shows a screenshot of a code editor with a Python script and its execution output in a terminal. The script defines three validation functions: validate_email, validate_password, and validate_mobile. It then prompts the user for email, password, and mobile number, and prints the validation results. The terminal output shows the user inputting 'charan123@gmail.com', 'Charan@123', and '9876543210', which are then validated as 'Valid Email', 'Strong Password', and 'Valid Mobile Number' respectively.

```
1 import re
2
3 def validate_email(email):
4     pattern = r'^[A-Za-z][A-Za-z0-9_]*@[A-Za-z]+\.(com|org|edu|net|in)$'
5     return "Valid Email" if re.fullmatch(pattern, email) else "Invalid Email"
6
7 def validate_password(password):
8     pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$%&]).{8,}$'
9     return "Strong Password" if re.fullmatch(pattern, password) else "Weak Password"
10
11 def validate_mobile(mobile):
12     pattern = r'^[6-9]\d{9}$'
13     return "Valid Mobile Number" if re.fullmatch(pattern, mobile) else "Invalid Mobile Number"
14
15 print("==== Intelligent Validator =====")
16
17 email = input("Enter Email: ")
18 password = input("Enter Password: ")
19 mobile = input("Enter Mobile Number: ")
20
15 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER TIMELINE + - 2: Python
PS C:\Charan\Simats\NLP\CO1 AT3\programs> & "C:\Users\Charan.kumar.Reddy\AppData\Local\Programs\Python\Python314\python.exe"
s" "c:/Charan/Simats/NLP/CO1 AT3/programs/1.py"
==== Intelligent Validator =====
Enter Email: charan123@gmail.com
Enter Password: Charan@123
Enter Mobile Number: 9876543210

----- Validation Result -----
Valid Email
Strong Password
Valid Mobile Number
PS C:\Charan\Simats\NLP\CO1 AT3\programs>
```

Explanation

The program uses Regular Expressions to validate the email address, password, and mobile number. It checks whether the email follows the required format, the password satisfies all security rules, and the mobile number contains exactly 10 digits starting with a digit between 6 and 9.

Result

The program successfully validates the email address, password, and mobile number using Regular Expressions and displays the appropriate validation results.

Question 2: Design a Finite State Automata (FSA) Simulator

Aim

To develop a Python program to simulate a Deterministic Finite Automata (DFA) that accepts strings ending with **"ab"** and displays the transition path and final result.

Algorithm

1. Define the DFA states and transition table.
2. Read the input string.
3. Process each character and move to the next state.
4. Display the transition path.
5. Check the final state and print whether the string is accepted or rejected.

Program

```
# DFA to accept strings ending with "ab"
```

```
transitions = {
```

```
    "q0": {"a": "q1", "b": "q0"},
```

```
    "q1": {"a": "q1", "b": "q2"},
```

```
    "q2": {"a": "q1", "b": "q0"}  
}
```

```
start_state = "q0"
```

```
final_state = "q2"
```

```
string = input("Enter Input String: ")
```

```
current = start_state
```

```
path = [current]
```

```
valid = True
```

```
for ch in string:
```

```
    if ch not in ["a", "b"]:
```

```
        valid = False
```

```
        break
```

```
    current = transitions[current][ch]
```

```
    path.append(current)
```


if not valid:

```
print("Invalid Input")
```

else:

```
print("Transition Path:", " -> ".join(path))
```

if current == final_state:

```
print("Accepted")
```

else:

```
print("Rejected")
```

Sample Input

Enter Input String: abaab

Sample Output

Transition Path: q0 -> q1 -> q0 -> q1 -> q1 -> q2

Accepted

Test Case 1

Input

ab

Output

Transition Path: q0 -> q1 -> q2

Accepted

Test Case 2

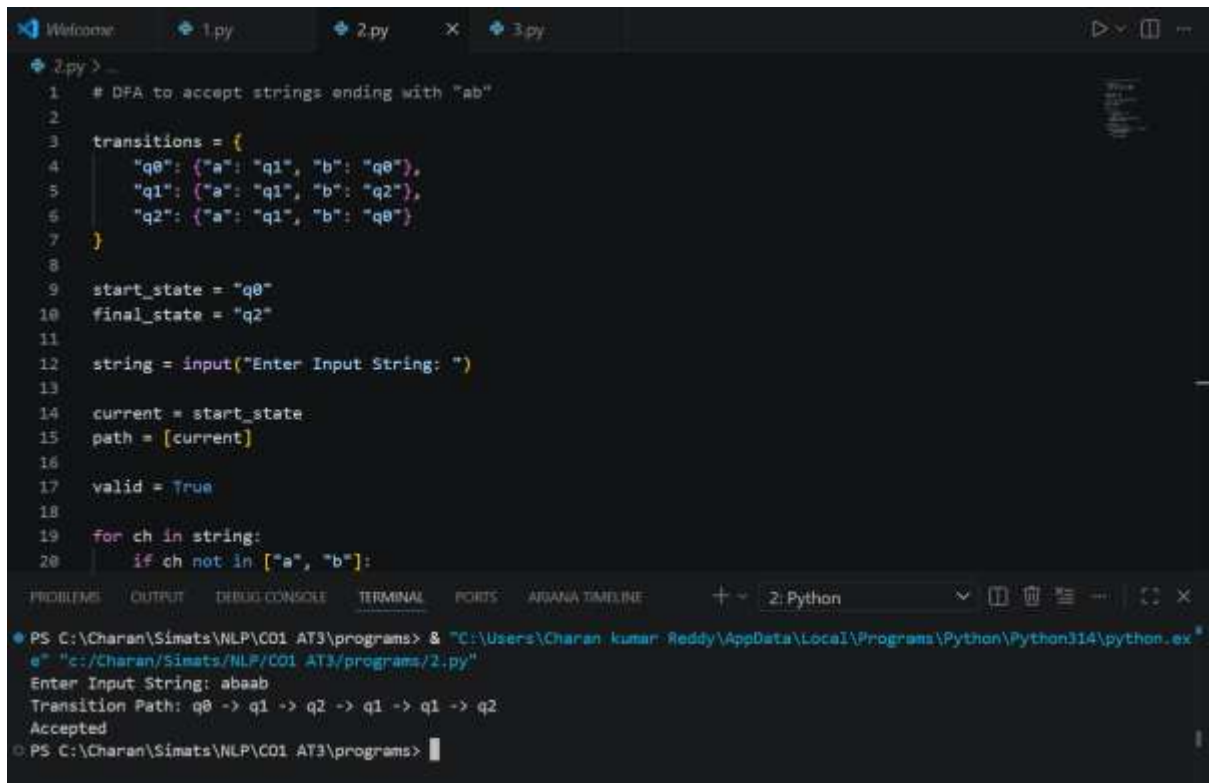
Input

abba

Output

Transition Path: q0 -> q1 -> q2 -> q0 -> q1

Rejected



```
1 # DFA to accept strings ending with "ab"
2
3 transitions = {
4     "q0": {"a": "q1", "b": "q0"},
5     "q1": {"a": "q1", "b": "q2"},
6     "q2": {"a": "q1", "b": "q0"}
7 }
8
9 start_state = "q0"
10 final_state = "q2"
11
12 string = input("Enter Input String: ")
13
14 current = start_state
15 path = [current]
16
17 valid = True
18
19 for ch in string:
20     if ch not in ["a", "b"]:
```

PS C:\Charan\Simats\NLP\CO1 AT3\programs> & "C:\Users\Charan.kumar.Reddy\AppData\Local\Programs\Python\Python314\python.exe" "c:\Charan\Simats\NLP\CO1 AT3\programs\2.py"

Enter Input String: abaab

Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2

Accepted

PS C:\Charan\Simats\NLP\CO1 AT3\programs> |

Explanation

The program simulates a Deterministic Finite Automata (DFA) that accepts strings ending with "ab". It processes each character of the input string, moves between states according to the transition table, displays the transition path, and checks whether the final state is the accepting state.

Result

The program successfully simulates the DFA, displays the transition path, and correctly determines whether the input string is Accepted or Rejected.

Question 3: Smart Pattern Matching Engine using Regular Expressions

Aim

To develop a Python program using Regular Expressions (Regex) to search different patterns such as dates, phone numbers, hashtags, mentions, prefixes, and suffixes from a given text.

Algorithm

1. Import the re module.
2. Read the input text and user choice.
3. Search the required pattern using Regular Expressions.
4. Display all matching results.
5. End the program.

Program

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")

choice = int(input("Enter your choice: "))

if choice == 1:
    print(re.findall(r"\d{2}/\d{2}/\d{4}", text))

elif choice == 2:
```

```
print(re.findall(r"[6-9]\d{9}", text))
elif choice == 3:
    print(re.findall(r"#\w+", text))
elif choice == 4:
    print(re.findall(r"@ \w+", text))
elif choice == 5:
    prefix = input("Enter Prefix: ")
    print(re.findall(r"\b" + prefix + r"\w*", text, re.IGNORECASE))
elif choice == 6:
    suffix = input("Enter Suffix: ")
    print(re.findall(r"\b\w*" + suffix + r"\b", text, re.IGNORECASE))
else:
    print("Invalid Choice")
```

Sample Input

Enter your choice: 3

Sample Output

['#NLP']

Test Case 1

Input

Choice: 1

Output

['12/09/2026']

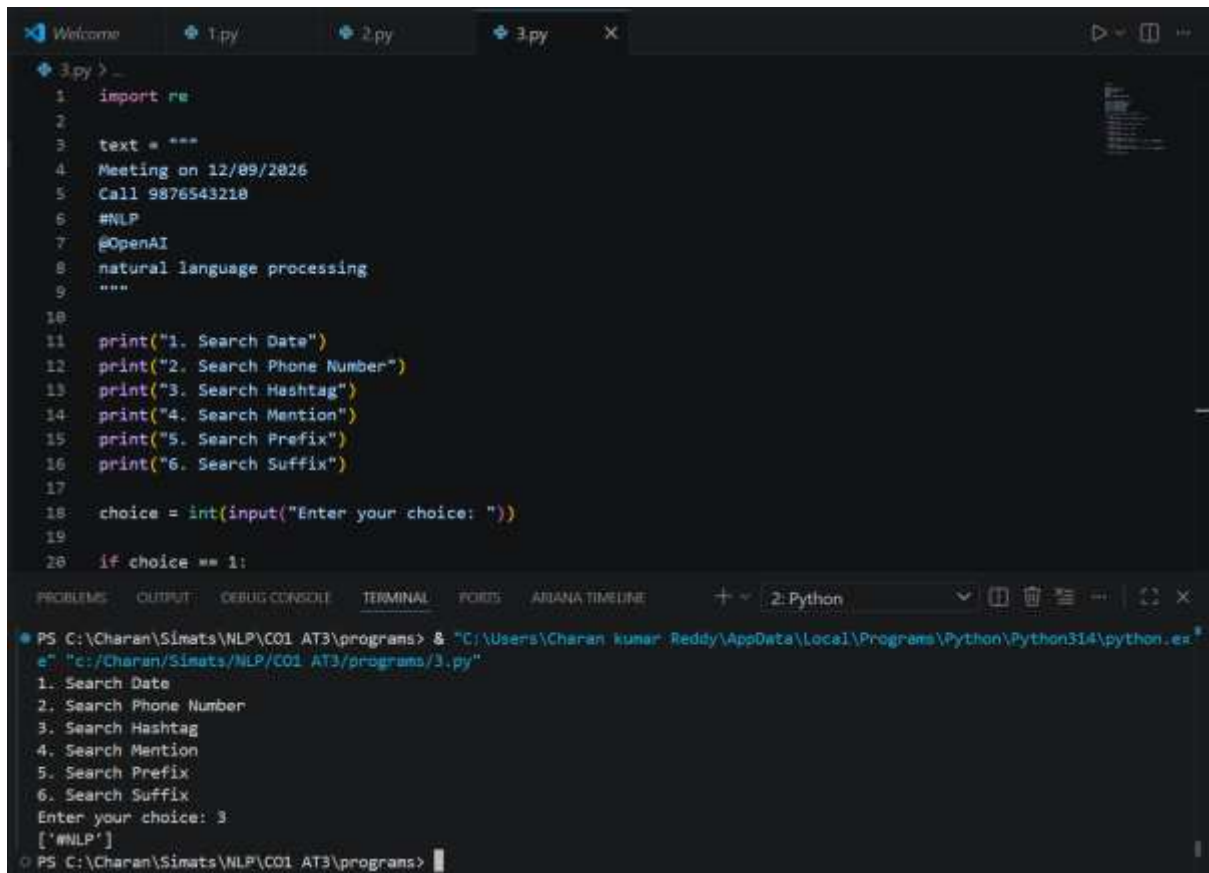
Test Case 2

Input

Choice: 2

Output

['9876543210']



```
1 import re
2
3 text = """
4 Meeting on 12/09/2026
5 Call 9876543210
6 #NLP
7 @OpenAI
8 natural language processing
9 """
10
11 print("1. Search Date")
12 print("2. Search Phone Number")
13 print("3. Search Hashtag")
14 print("4. Search Mention")
15 print("5. Search Prefix")
16 print("6. Search Suffix")
17
18 choice = int(input("Enter your choice: "))
19
20 if choice == 1:
```

```
PS C:\Charan\Simats\NLP\COI AT3\programs> & "C:\Users\Charan kumar Reddy\AppData\Local\Programs\Python\Python314\python.exe"
e" "c:\Charan\Simats\NLP\COI AT3\programs\3.py"
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
Enter your choice: 3
['#NLP']
PS C:\Charan\Simats\NLP\COI AT3\programs>
```

Explanation

The program uses Regular Expressions to search different patterns such as dates, phone numbers, hashtags, mentions, prefixes, and suffixes from the given text. Based on the user's choice, it displays all matching patterns.

Result

The program successfully performs pattern matching using Regular Expressions and displays the required search results according to the user's selection.